National Institute of Technology Warangal
Department of Computer Science and Engineering

Course Project Report: Advanced Computer Networks
Project ID 22

Reproducing Cross-Domain Lifelong Learning for SDN-Based Configuration and Resource Allocation in Satellite–Airborne–Terrestrial Edge Computing Networks

Implementation and experimental study of
"Lifelong Learning-Based SDN Design for Dynamic Configuration and Resource Allocation in Satellite-Terrestrial Networks"
(Wu *et al.*, IEEE Transactions on Wireless Communications, 2026)

Submitted by

Vedant Amit Agrawal	23CSB0A08
Abu Fahad Ansari	23CSB0A45
Shwetang Wasnik	23CSB0A55

Under the guidance of
Dr. Venkatarami Reddy Chintapalli
Department of Computer Science and Engineering

November 2026

Abstract

Satellite–airborne–terrestrial edge computing networks (SATECNs) let Internet-of-Things (IoT) devices compute locally or offload work to unmanned aerial vehicles (UAVs) and low-Earth-orbit (LEO) satellites. Because the topology and the load change all the time, an offloading controller has to keep re-learning, and an ordinary reinforcement-learning (RL) agent forgets old situations whenever it learns new ones. Wu et al. [1] address this with a hierarchical software-defined-networking (SDN) control plane whose global controller maintains a shared knowledge base built by cross-domain lifelong learning (CDLL): every task policy is factorised as $\theta^k = \Psi^g \mathbf{L}\mathbf{s}^k$, a domain projection times a shared basis times sparse task coefficients, so that tasks whose action spaces have different dimensions can still share knowledge.

This project re-implements the whole pipeline from scratch in Python: a slotted SATECN simulator driven by real Starlink two-line elements propagated with SGP4, the six RL domains of the paper, the episodic natural actor–critic (NAC) and the multi-step-return MSR-NAC base learners, the single-domain lifelong baseline (MSR-NAC-ELLA), the proposed MSR-NAC-CDLL, and an exhaustive grid-search reference. We reproduce every experiment of the paper's evaluation section (convergence, per-domain penalty, AoI and energy, the β trade-off, jumpstart, convergence speed, system analysis, scalability and continual learning) and add ablations. Our simulator reproduces the paper's LEO visibility statistic exactly (on average 2.73 satellites visible above 55°). The learning results reproduce some of the paper's qualitative findings but not its large margins; Chapter 7 reports what we measured, and Chapter 8 explains where and why the numbers differ. Every parameter that the paper does not specify is listed as an explicit assumption.

Contents

1 Introduction **6**

1.1 Background 6
1.2 Motivation 6
1.3 Objectives of this project 6
1.4 Report organisation 7

2 Literature Review **8**

2.1 AoI-aware offloading in non-terrestrial networks 8
2.2 SDN for space–air–ground networks 8
2.3 Lifelong and continual reinforcement learning 8
2.4 Research gap 8

3 Problem Statement and System Model **10**

3.1 Network architecture 10
3.2 SAT configurations and domains 11
3.3 Computing model 11
3.4 Communication model 11
3.5 Queues and age of information 12
3.6 Optimisation problem 12

4 Proposed Methodology **13**

4.1 Policy-gradient objective 13
4.2 Cross-domain factorisation 13
4.3 Second-order approximation and updates 14
4.4 Base learner: MSR-NAC 14
4.5 Algorithm 14

5 Implementation **16**

5.1 Technology stack 16
5.2 Code architecture 16
5.3 LEO geometry from real ephemerides 17
5.4 Simulator 17
5.5 Policy parameterisation 17
5.6 Learners 18

5.7 Testing 18

6 Experimental Setup **20**
6.1 Parameters 20
6.2 Task stream and protocol 21
6.3 Evaluation metrics 21

7 Results and Discussion **22**
7.1 Convergence in domain 6 (Fig. 3 of the paper) 22
7.2 Per-domain penalty, AoI and energy (Fig. 4 of the paper) 23
7.3 Convergence speed (Fig. 7 of the paper) 25
7.4 Exhaustive-search reference 25
7.5 AoI–energy trade-off versus β (Fig. 5 of the paper) 26
7.6 Jumpstart on unseen tasks (Fig. 6 of the paper) 27
7.7 System analysis with the trained CDLL policy (Fig. 8 of the paper) 28
7.8 Scalability and control-plane overhead (Table V of the paper) 29
7.9 Continual learning and forgetting (Fig. 9 of the paper) 29
7.10 Ablation study 30

8 Comparison with the Paper and Limitations **32**
8.1 Side-by-side comparison 32
8.2 Why our numbers differ 32
8.3 Limitations of this study 33

9 Conclusion and Future Scope **34**
9.1 Conclusion 34
9.2 Future scope 34

A Reproducing the Results **38**

List of Figures

3.1 Hierarchical KB-SAT-SDN architecture as implemented (redrawn after Fig. 1 of Wu et al. [1]). 10

4.1 The CDLL policy factorisation of Eq. (20). 13
4.2 CDLL-SATECN workflow (redrawn after Fig. 2 of Wu et al. [1]). 15

5.1 Module structure of the implementation. 16
5.2 Terminal output of the unit tests and of the full experiment run (log excerpt). . . 19

7.1 Ours: average penalty vs. iteration on the domain-6 tasks (mean ± std over 3 seeds, 5-iteration moving average). 22
7.2 Ours: average penalty, AoI and scaled energy per domain over 200 iterations (error bars: std over seeds). 23
7.3 Ours: convergence speedup relative to NAC (iterations needed to reach NAC's final penalty level). 25
7.4 Ours: MSR-NAC-CDLL trained on the domain-6 tasks for each β (100 iterations, seed 0). Shaded band: the paper's recommended range $\beta \in [0.8, 0.9]$. 26
7.5 Ours: initial-penalty reduction relative to MSR-NAC (random initialisation) after 15 pre-training rounds on tasks 1–24. 27
7.6 Ours: penalty and AoI, task distribution and energy breakdown per domain with the trained CDLL policies (5 evaluation episodes per task, 3 seeds). 28
7.7 Ours: learning four new domain-6 tasks (25–28) in a randomised sequence of 32 encounters after pre-training on tasks 1–24. 29
7.8 Ours: ablations. 31

List of Tables

3.1 RL domains (Table II of Wu et al. [1]) and the policy dimension of our implementation. 11

5.1 Modules and the parts of the paper they implement. 17

6.1 Simulation parameters (P = paper, A = assumed). 20

7.1 Average penalty over the 200 training iterations (mean over 3 seeds; lowest per domain in bold). 23
7.2 Average AoI (slots) over the 200 training iterations. 23
7.3 Average scaled energy over the 200 training iterations. 24
7.4 Penalty at the end of training (mean of the last 10 iterations, 3 seeds). 24
7.5 Convergence speedup relative to NAC (NAC = 1 by definition). 25
7.6 AoI and scaled energy of MSR-NAC-CDLL on the domain-6 tasks for different β. 26
7.7 Jumpstart: initial-penalty reduction (%) on unseen tasks relative to MSR-NAC from a random start. 27
7.8 System analysis with the trained CDLL policies (per 120-slot episode; tasks generated and completed by executor). 28
7.9 Scalability with $U = N/5$ UAVs and 18 LEOs over 600 slots (3 seeds). Control-plane figures are modelled, not measured. 29
7.10 Continual learning on new domain-6 tasks 25–28 and forgetting on old tasks 1–24 (mean penalty, 3 seeds). 30
7.11 Ablations (mean penalty over training and over the last 10 iterations, seed 0, 100 iterations). 30

8.1 The paper's claims against our measurements. 32

Chapter 1

Introduction

1.1 Background

Sixth-generation (6G) networks are expected to serve IoT applications everywhere, including farms, oceans and disaster zones where terrestrial infrastructure is thin. Two kinds of non-terrestrial node fill the gap: UAVs that hover tens of metres above the devices and can host a small edge server, and LEO satellites such as Starlink that orbit at roughly 500 km and are visible from any point for a few minutes per pass. A network that combines the three layers is called a satellite–airborne–terrestrial edge computing network (SATECN).

Many of the applications that motivate SATECNs (crop monitoring, industrial sensing, alarms) are *time-sensitive*: what matters is how fresh the processed information is. Freshness is measured by the *age of information* (AoI) [2], the time elapsed since the newest piece of processed data was generated. Lowering AoI means processing tasks quickly, which costs CPU cycles and transmit energy. The controller therefore trades AoI against energy, and the best trade-off depends on which nodes are reachable, how loaded they are and how good the radio links are, all of which change continually.

1.2 Motivation

Classical optimisation re-solves the problem whenever conditions change, which is too slow at slot level. Deep RL learns a policy for one environment but, when the environment changes, either relearns from scratch or suffers catastrophic forgetting [3]. Lifelong learning [4] keeps a knowledge base that grows with every task. SATECNs add a further difficulty: an IoT device that can only compute locally faces a different action space from one that can choose between a UAV and a satellite and also set its transmit power. Standard lifelong and continual learners assume one fixed action dimension.

1.3 Objectives of this project

1. Build a working, reproducible implementation of the KB-SAT-SDN system model and the CDLL-SATECN algorithm of Wu et al. [1].

2. Validate each component against the paper's equations.
3. Re-run the paper's experiments (Figs. 3–9 and Table V of Wu et al. [1]) and compare our numbers with the reported ones without tuning towards them.
4. Document every assumption and deviation so that the study can be repeated and criticised.

1.4 Report organisation

Chapter 2 reviews related work. Chapter 3 states the problem and the system model, Chapter 4 the learning method, Chapter 5 the implementation, Chapter 6 the experimental setup, Chapter 7 the results, Chapter 8 the comparison with the paper and the limitations, and Chapter 9 concludes.

Chapter 2

Literature Review

2.1 AoI-aware offloading in non-terrestrial networks

Xia *et al.* [5] studied federated computation offloading across satellite, airborne and terrestrial layers to minimise AoI. Liao *et al.* [6] optimised peak AoI for LEO-satellite IoT with freshness- and fairness-aware scheduling. Zhu *et al.* [7] proposed federated multi-agent actor–critic learning for age-sensitive mobile edge computing and supplied the air-to-ground channel model also used here. These works optimise a single, fixed configuration and do not address learning across changing configurations.

2.2 SDN for space–air–ground networks

SDN separates the control plane from the data plane [8]. Shi *et al.* [9] and Chen *et al.* [10] proposed multi-layer and hierarchical controller placements for space–terrestrial networks. In those works the controllers make placement or routing decisions; they do not learn offloading policies or share learned knowledge.

2.3 Lifelong and continual reinforcement learning

PG-ELLA [4] represents each task's policy as $\theta^k = \mathbf{L}\mathbf{s}^k$ and updates $\mathbf{L}$ online from a second-order approximation of each task's objective, so the cost of a new task does not grow with the number of earlier tasks. Its cross-domain extension [11] adds a projection Ψ^g per domain so that tasks with different state or action dimensions can share the same $\mathbf{L}$. The authors' preliminary work [12] applied PG-ELLA to SATECNs. Continual-learning methods for neural networks such as EWC [3], GEM [13] and A-GEM [14] protect old tasks through penalties or replay but assume a fixed network shape.

2.4 Research gap

No earlier work combines (i) a hierarchical SDN control plane for all three layers, (ii) AoI–energy-optimal offloading and resource allocation, and (iii) knowledge transfer across tasks whose action spaces differ in dimension. Wu et al. [1] fill this gap with CDLL-SATECN. The gap this

project addresses is reproducibility: the paper gives no code, and several modelling and learning parameters are not stated.

Chapter 3

Problem Statement and System Model

This chapter restates the model of Wu et al. [1] in our own words and notation; equation numbers in parentheses refer to the paper.

3.1 Network architecture

Figure 3.1 shows the hierarchical KB-SAT-SDN architecture. Local controllers for the satellite (S-SDNC), airborne (A-SDNC) and terrestrial (T-SDNC) segments report segment state to a global SAT-SDNC hosted on a high-altitude platform. The SAT-SDNC identifies RL tasks and domains, runs the lifelong-learning update and pushes the resulting policies back down. Execution is distributed: every IoT applies its policy locally in each 1 s slot.

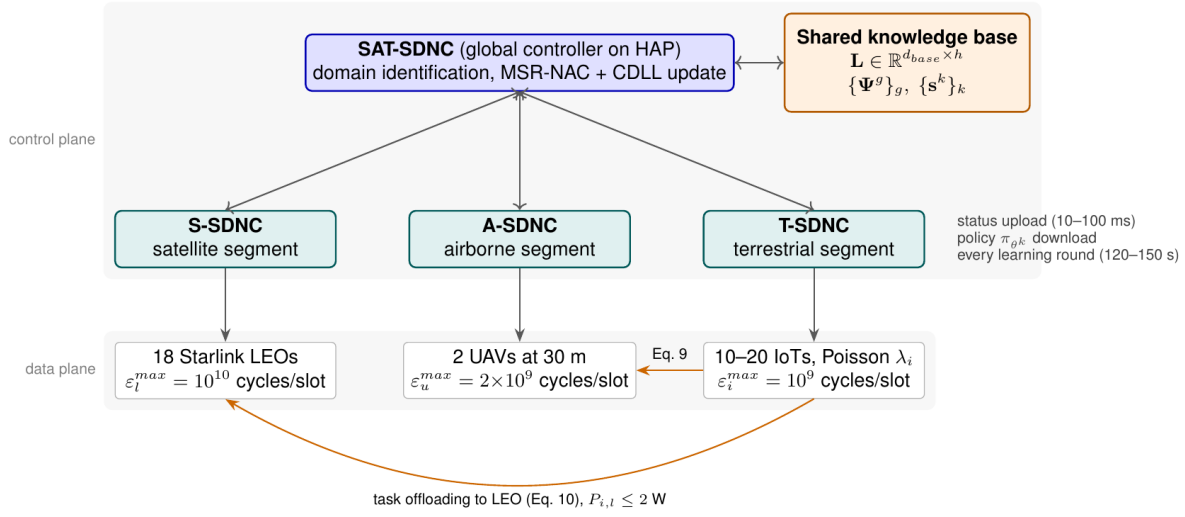


Figure 3.1: Hierarchical KB-SAT-SDN architecture as implemented (redrawn after Fig. 1 of Wu et al. [1]).

3.2 SAT configurations and domains

Each task v is executed by exactly one node: locally, by a UAV, or by a LEO satellite. With binary indicators $y_i^{v,c}$ (local) and $y_{i,o}^{v,c}$ (offload to node o) for configuration c,

$$\sum_{c\in\mathcal{C}} \left(y_i^{v,c} + y_{i,u}^{v,c} + y_{i,l}^{v,c} \right) \le 1. \tag{1}$$

The paper groups the seven configurations of its Table I into six RL domains (Table II) according to which decisions are being learned. Table 3.1 lists them together with the policy dimension d^g produced by our parameterisation (Section 5.5).

Table 3.1: RL domains (Table II of Wu et al. [1]) and the policy dimension of our implementation.

Domain	Config.	Decisions learned	IoTs	Nodes	d^g
1	1	local hold/compute, CPU share ε	10	–	6
2	2, 3	transmit power (10 IoTs fixed to UAVs, 10 to LEOs)	20	2 UAV + 18 LEO	6
3	6	UAV vs. LEO (fixed power)	10	2 UAV + 18 LEO	3
4	6	UAV vs. LEO, transmit power	10	2 UAV + 18 LEO	6
5	4	local vs. UAV, ε, UAV CPU share, power	10	2 UAV	12
6	4, 5, 7	local vs. UAV vs. LEO, ε, remote CPU share, power	10	2 UAV + 18 LEO	15

3.3 Computing model

A task of IoT i generated in slot t has size $D_i^v(t)$ bits and needs $\varepsilon^v = \rho D_i^v(t)$ CPU cycles (2). With f cycles per second allocated, the computing delay is ε^v/f (3) and the energy is

$$E^{\text{cpt}} = \alpha\, \varepsilon^v f^2 \tag{4}$$

with hardware constant α (10^{-25} for IoTs and UAVs, 10^{-28} for LEOs). Offloading takes D/R seconds of transmission (5) and costs $P\,D/R$ joules (6). The total energy attributed to IoT i is the sum of local computing, transmission and remote computing energy, weighted by the scaling factors ξ of Table IV (7).

3.4 Communication model

IoT–UAV. The average path loss mixes line-of-sight (LoS) and non-LoS components,

$$L_{i,u} = p_0 PL_0 + (1-p_0) PL_1, \qquad R_{i,u} = B \log_2\!\Big(1 + \frac{P_{i,u}}{L_{i,u} N_0 B}\Big), \tag{8–9}$$

where $p_0 = 1/(1 + a\, e^{-b(\vartheta - a)})$ depends on the elevation angle ϑ [15].

IoT–LEO. The uplink rate is $R_{i,l} = B \log_2(1 + P_{i,l}\, g_i H_{i,l}/N_0)$ (10), where $H_{i,l}$ includes free-space loss over the slant range and Rician fading. The slant range and the visibility of every satellite come from real Starlink ephemerides (Section 5.3).

3.5 Queues and age of information

In every slot the data an IoT offloads is limited by its link, $D_{i,o}^{\mathrm{tr}} = \min(\sum D_i^{\upsilon}, R_{i,o}\Delta t)$ (11), and the backlog evolves as $D_i^{\mathrm{rem}}(t+1) = \max\{0, D_i^{\mathrm{rem}} + D_i^{\mathrm{new}} - D_i^{\mathrm{cpt}} - D_{i,o}^{\mathrm{tr}}\}$ (12). The AoI grows by one per slot and drops when a task completes:

$$\Delta_i(t+1) = \begin{cases} \Delta_i(t) + 1, & \text{no task of } i \text{ completes in slot } t, \\ t + 1 - t_i^{\mathrm{gen},\upsilon^*}, & \text{otherwise.} \end{cases} \tag{14}$$

3.6 Optimisation problem

The per-slot penalty of IoT i is

$$C_i(t) = \beta\, \Delta_i(t) + \epsilon(1-\beta)\Big(E_i(t) + \sum_o E_{i,o}(t)\Big), \tag{15}$$

and the controller minimises its long-run average over offloading decisions, CPU allocations and transmit powers subject to (1)–(14), CPU capacities (16c–d) and power limits (16e). Because the environment is non-stationary, the paper recasts (16) as a sequence of RL tasks, each an MDP whose state is $x_t = \{\Delta(t), D^{\mathrm{rem}}(t)\}$ and whose cost is $C(t)$.

Chapter 4

Proposed Methodology

4.1 Policy-gradient objective

For a task k the agent minimises the expected penalty of a trajectory, $\mathcal{J}_k(\theta^k) = \int p_{\theta^k}(\tau)\Re^k(\tau)\,d\tau$. Over K tasks the lifelong objective is min $\frac{1}{K}\sum_k \mathcal{J}_k(\theta^k) + \mathrm{Reg}(\Theta)$ (19).

4.2 Cross-domain factorisation

CDLL writes every task policy as

$$\theta^k = \underbrace{\Psi^{g(k)}}_{d^g \times d_{\text{base}}} \underbrace{\mathbf{L}}_{d_{\text{base}} \times h} \underbrace{\mathbf{s}^k}_{h \times 1}, \tag{20}$$

(Figure 4.1). $\mathbf{L}$ is the domain-agnostic knowledge base shared by all tasks, Ψ^g maps it into the parameter space of domain g, and $\mathbf{s}^k$ is a sparse code that specialises the policy to task k. A new task in a known domain only needs its h-dimensional code; a new domain additionally needs its projection.

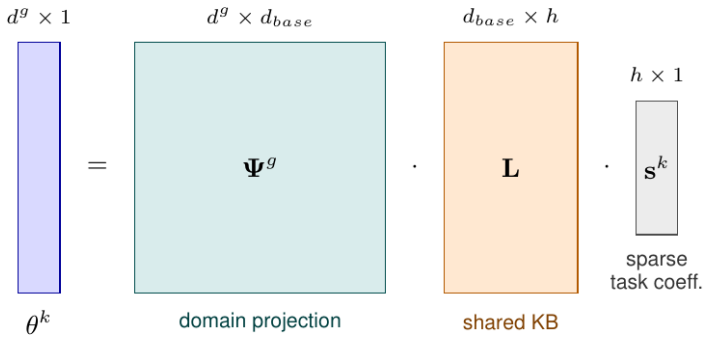


Figure 4.1: The CDLL policy factorisation of Eq. (20).

4.3 Second-order approximation and updates

Optimising (19) directly would need every old trajectory. Following PG-ELLA [4, 11], the paper expands $\mathcal{J}_k$ to second order around the single-task estimate $\hat{\theta}^k$ with Hessian $\mathbf{H}^k$:

$$\hat{e}_T(\mathbf{L}, \Psi^1, \ldots, \Psi^G) = \sum_g \frac{1}{|\mathcal{K}^g|} \sum_{k \in \mathcal{K}^g} \min_{\mathbf{s}^k} \Big[\|\hat{\theta}^k - \Psi^g \mathbf{L}\mathbf{s}^k\|^2_{\mathbf{H}^k} + \mu_1 \|\mathbf{s}^k\|_1 \Big] + \mu_2 \|\Psi^g\|_F^2 + \mu_3 \|\mathbf{L}\|_F^2. \quad (22)$$

Differentiating gives the gradient steps

$$\Delta \mathbf{L} = \eta_L \Big[\sum_g \frac{1}{|\mathcal{K}^g|} \sum_k \big(- \Psi^{g\top} \mathbf{H}^k \hat{\theta}^k \mathbf{s}^{k\top} + \Psi^{g\top} \mathbf{H}^k \Psi^g \mathbf{L}\mathbf{s}^k \mathbf{s}^{k\top} \big) + \mu_3 \mathbf{L} \Big], \quad (23)$$

$$\Delta \Psi^g = \eta_\Psi \Big[\frac{1}{|\mathcal{K}^g|} \sum_k \big(- \mathbf{H}^k \hat{\theta}^k (\mathbf{L}\mathbf{s}^k)^\top + \mathbf{H}^k \Psi^g \mathbf{L}\mathbf{s}^k (\mathbf{L}\mathbf{s}^k)^\top \big) + \mu_2 \Psi^g \Big], \quad (24)$$

and the code is the solution of a weighted Lasso [16],

$$\mathbf{s}^{k*} = \arg\min_{\mathbf{s}} \|\hat{\theta}^k - \Psi^g \mathbf{L}\mathbf{s}\|^2_{\mathbf{H}^k} + \mu_1 \|\mathbf{s}\|_1. \quad (25)$$

4.4 Base learner: MSR-NAC

The paper extends the episodic natural actor–critic [17] with n-step returns [18]. Our version (Section 5.6) fits a critic $V(x)$, forms n-step targets $G_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j C_{t+j} + \gamma^n V(x_{t+n})$, and regresses the advantage $G_t^{(n)} - V(x_t)$ on the compatible features $\nabla_\theta \log \pi_\theta(a_t|x_t)$. With compatible features the regression weights $\mathbf{w}$ equal the natural gradient $F^{-1}\nabla_\theta \mathcal{J}$ [17, 19]. The episodic NAC baseline instead regresses whole-trajectory returns on the summed score $\sum_t \gamma^t \nabla \log \pi$.

4.5 Algorithm

Algorithm 1 is Algorithm 1 of the paper as implemented; Figure 4.2 shows how it runs inside the SDN hierarchy.

Algorithm 1: MSR-NAC-CDLL as implemented (after Algorithm 1 of Wu et al. [1])

Input: config, task stream, h, d_{base}, $\mu_{1,2,3}$, η_L, η_Ψ

Initialise $\mathbf{L} \sim \mathcal{N}(0, 1/h)$;
for *iteration* $= 1, \ldots, 200$ **do**
  foreach *task* k *in a random order* **do**
    if k *is new* **then**
      if *domain* $g(k)$ *is new* **then** $\Psi^g \sim \mathcal{N}(0, 1/d_{\text{base}})$;
      $\mathbf{s}^k \leftarrow$ mean code of known tasks (*deviation*; Alg. 1 uses random);
    $\theta^k \leftarrow \Psi^g \mathbf{L} \mathbf{s}^k$; roll out one 120-slot episode with all IoTs of task k;
    $(\mathbf{w}, F) \leftarrow$ MSR-NAC regression; $\hat{\theta}^k \leftarrow \theta^k - \eta\, \mathbf{w}/\sqrt{\mathbf{w}^\top F \mathbf{w}}$;
    $\mathbf{H}^k \leftarrow F/\bar{\lambda}(F) + \lambda I$;
    repeat
      $\mathbf{L} \leftarrow \mathbf{L} - \frac{\eta_L}{\text{Lip}_L} \nabla_{\mathbf{L}} \hat{e}_T$ (Eq. 23); $\Psi^g \leftarrow \Psi^g - \frac{\eta_\Psi}{\text{Lip}_\Psi} \nabla_{\Psi^g} \hat{e}_T$ (Eq. 24)
    until *10 times*;
    re-solve the Lasso (25) for $\mathbf{s}^{k'}$ of every stored task k' by coordinate descent;

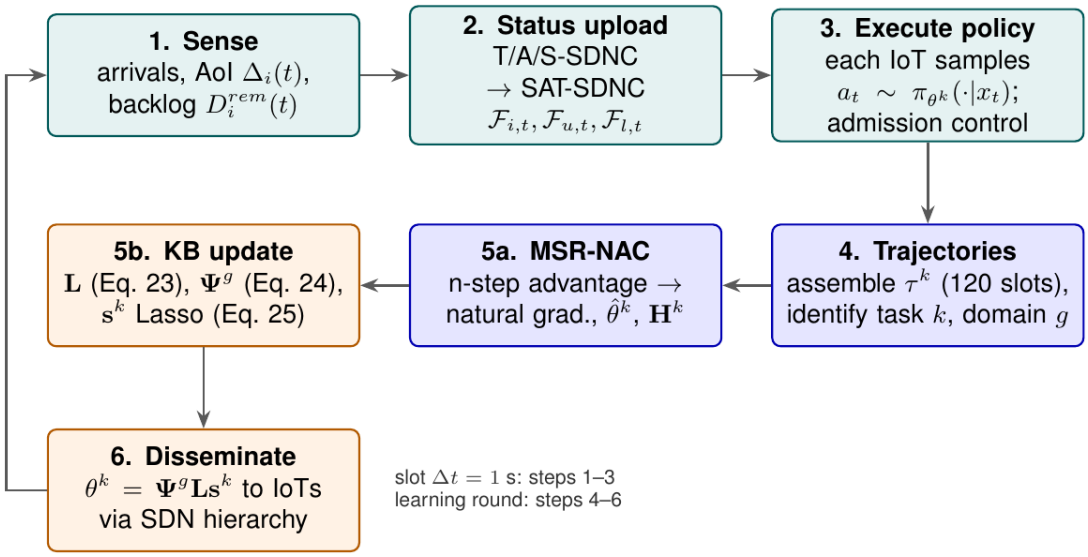


Figure 4.2: CDLL-SATECN workflow (redrawn after Fig. 2 of Wu et al. [1]).

Chapter 5

Implementation

5.1 Technology stack

The paper reports only "Python simulations". The models are linear policies with at most 15 parameters, so no deep-learning framework or GPU is needed. We used Python 3 with NumPy [20] for all numerics, Matplotlib [21] for figures, PyYAML for configuration and `sgp4` [22, 23] for orbit propagation, and `multiprocessing` to run independent training jobs in parallel. A single YAML file holds every parameter, each tagged `[paper]` or `[assumed]`. Everything runs from the command line; no web interface is needed for a simulation study.

The source code is hosted at `https://github.com/AnsFahad/satecn-cdll`.

5.2 Code architecture

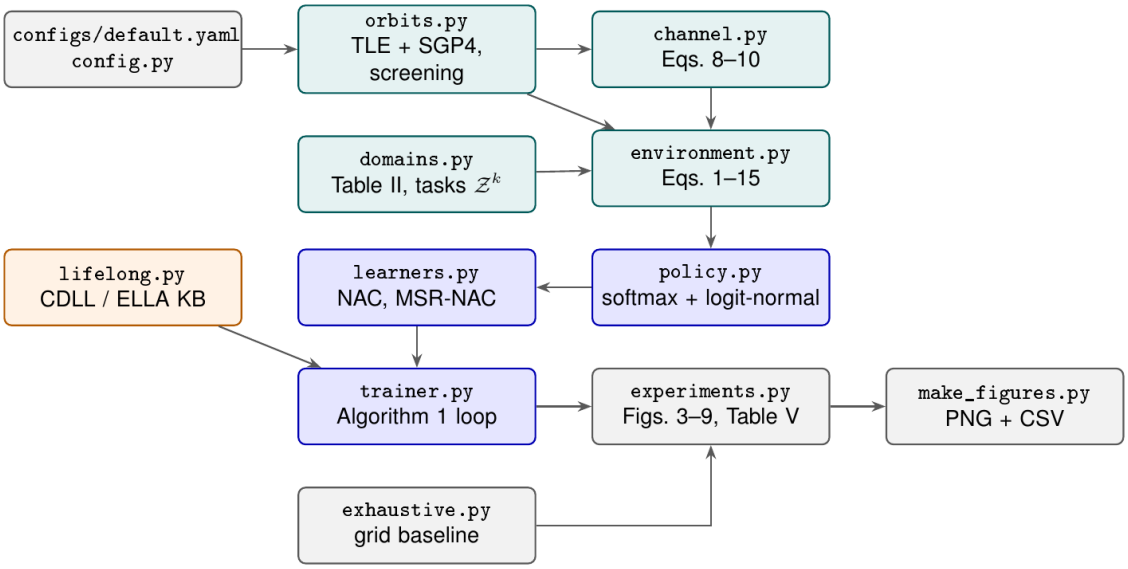


Figure 5.1: Module structure of the implementation.

Table 5.1: Modules and the parts of the paper they implement.

Module	Content
`orbits.py`	TLE loading, SGP4 propagation, TEME→ECEF, elevation/slant range, the 55°/90 s screening of Sec. V-A; synthetic fallback
`channel.py`	Air-to-ground LoS/NLoS path loss and rate (8)–(9); LEO uplink with Rician fading (10)
`domains.py`	Table II domains, policy dimensions, task sampling (17)
`environment.py`	Slotted simulator: arrivals, dispatch with (1), local/remote computing (2)–(4), transmission (5)–(6), (11), queues (12), AoI (14), penalty (15)
`policy.py`	Softmax executor head plus logit-normal resource heads; score function $\nabla \log \pi$
`learners.py`	Roll-outs, episodic NAC, MSR-NAC, Fisher/Hessian, fixed-KL natural step
`lifelong.py`	Knowledge base for CDLL (shared $\mathbf{L}$, per-domain Ψ^g, (23)–(25)) and ELLA (per-domain $\mathbf{L}^g$)
`trainer.py`	Algorithm 1 loop, interleaved task stream
`exhaustive.py`	Grid search over stationary decisions
`experiments.py`	One function per experiment of Sec. V

5.3 LEO geometry from real ephemerides

The paper takes satellite visibility from public Starlink TLEs. We downloaded a CelesTrak snapshot [24] (10,623 satellites, epoch around 7–8 November 2026), propagated it with SGP4 over a 600 s window for a ground site at 42.37°N, 72.52°W, and kept the satellites that stay at or above 55° elevation for at least 90 consecutive seconds. Scanning window start times, the first window that gave exactly 18 satellites (start 2026-11-08 00:30 UTC) also gives an average of **2.73 satellites visible per slot, the same value the paper reports**. **[Deviation]** Since the paper was written, Starlink has lowered most shells to about 470–490 km, so the paper's 520–580 km altitude filter now keeps almost nothing; we widened it to 440–600 km. The 18 screened TLEs ship with the code in `data/starlink_screened.txt`.

5.4 Simulator

Each slot runs, for every IoT: Poisson arrivals with sizes drawn around the IoT's mean D_i; dispatch of head-of-line tasks to the executor chosen by the policy, in first-come-first-served order, binding each task to one executor (no splitting); local computing with the allocated share of $\varepsilon_i^{\max}$; remote computing with admission scaling, so that the CPU shares requested at a UAV or LEO never exceed its capacity (16d); uplink transmission over a single radio to the best visible node; and finally the AoI and penalty updates. A task that finishes uploading starts computing in the next slot.

5.5 Policy parameterisation

The paper says only that policies are compact and live in a normalised space. We use features $\phi(x) = [1, \Delta/50, D^{\text{rem}}/20\,\text{MB}]$. The executor choice is a softmax with logits linear in ϕ (the first category is the reference). Each continuous resource (CPU share, remote CPU share, power share) is logit-normal, $z \sim \mathcal{N}(\mathbf{w}^\top \phi, \sigma^2)$ with $a = \text{sigmoid}(z) \in (0, 1)$. This yields the dimensions of Table 3.1: from 3 (domain 3) to 15 (domain 6). **[Deviation]** We first used clipped Gaussians

and found that once a mean left $[0, 1]$ the clipped region gave no gradient and learning stalled; the logit-normal head fixed this.

5.6 Learners

MSR-NAC. The critic is linear in quadratic features of (Δ, D^{rem}), remaining-horizon terms (each trajectory window is finite) and per-pipeline backlogs. The actor still sees only $x = \{\Delta, D^{\text{rem}}\}$. Advantages $G_t^{(n)} - V(x_t)$ with $n = 40$ and $\gamma = 1$ are regressed on the score vectors by ridge regression. The update is a fixed-KL natural step $\theta \leftarrow \theta - \eta\, \mathbf{w}/\sqrt{\mathbf{w}^\top F \mathbf{w}}$ with $\eta = 0.2$ decaying by 0.995 per visit.

Hessian. $\mathbf{H}^k$ is the Fisher information of the roll-out, scaled to unit mean eigenvalue, plus $10^{-2} I$.

Knowledge base. $d_{\text{base}} = 15$ and $h = 8$. Eqs. (23)–(24) are applied ten times per task visit, with the step divided by a Lipschitz bound of the gradient: **[Deviation]** without this normalisation the bilinear updates diverged in our first runs. After each update the Lasso (25) is re-solved for all stored tasks by coordinate descent. **[Deviation]** Algorithm 1 updates only the current $\mathbf{s}^k$; without the refit, changes to the shared $\mathbf{L}$ silently moved the policies of tasks that had not been visited, which hurt performance (see the ablation in Section 7.10).

ELLA baseline. The same code with a separate $\mathbf{L}^g \in \mathbb{R}^{d^g \times h}$ per domain and no projection, i.e. PG-ELLA run separately in each domain.

Exhaustive search. Every stationary decision on a grid is simulated: executor probabilities on a simplex grid with step 0.25, and resource levels in $\{0.25, 0.5, 0.75, 1\}$ (960 points in domain 6). The best point is then re-evaluated on three fresh episodes.

5.7 Testing

Five unit tests (`tests/test_core.py`) check the domain dimensions; simulator invariants (AoI never grows by more than one per slot, backlog is non-negative, no more tasks complete than are generated); the policy score function against central finite differences; the Lasso solution against its KKT conditions; and that the CDLL knowledge base recovers policies generated from a shared low-rank basis. All five pass. Figure 5.2 shows the test run and the experiment runner's log from the run that produced the results in Chapter 7.

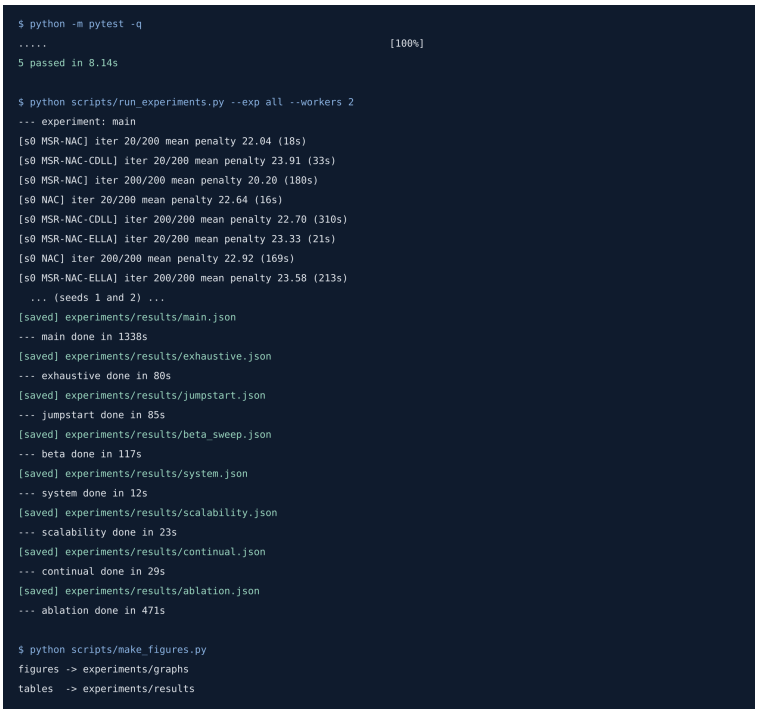


Figure 5.2: Terminal output of the unit tests and of the full experiment run (log excerpt).

Chapter 6

Experimental Setup

6.1 Parameters

Table 6.1 lists the parameters. Values from Table IV of Wu et al. [1] are used unchanged; the rest are our assumptions.

Table 6.1: Simulation parameters (P = paper, A = assumed).

Parameter	Value	Src.
Area, slot, episode	1000×1000 m, $\Delta t = 1$ s, 120 slots per trajectory window	P
Task size D_i	$\mathcal{N}(1.2, 0.6^2)$ MB per IoT (truncated at 0.2 MB); per task $\mathcal{N}(D_i, (0.1D_i)^2)$	P / A
Arrival rate λ_i	$U[6, 20] \times 0.05$ tasks/s (i.e. 0.3–1.0)	P / A
ρ	$U[180, 284]$ cycles/bit	P
$\varepsilon^{\max}$ IoT / UAV / LEO	10^9 / 2×10^9 / 10^{10} cycles/slot	P
α IoT, UAV / LEO	10^{-25} / 10^{-28}	P
ξ^{cpt} IoT / UAV / LEO, ξ^{tr}	1.0 / 0.75 / 0.6, 1.0	P / A
$P^{\max}$ to UAV / LEO	0.1 W / 2 W	P
Bandwidth	1 MHz per IoT	P
UAVs	2 at 30 m, random waypoint at 5–15 m/s	P / A
A2G channel	2 GHz, $\eta_{\text{LoS}} = 1$ dB, $\eta_{\text{NLoS}} = 20$ dB, $a = 9.61$, $b = 0.16$, NF 10 dB	A
LEO channel	2 GHz, satellite gain 22 dBi, NF 3 dB, Rician $K = 10$	A
β, ϵ	0.8, 1.0	P / A
Policy	$\sigma = 0.5$ (logit space), init $\theta \sim U(0, 1)$	A
MSR-NAC	$n = 40$, $\gamma = 1$, $\eta = 0.2$, decay 0.995, 1 episode per iteration	A
CDLL	$d_{\text{base}} = 15$, $h = 8$, $\mu_1 = 10^{-2}$, $\mu_2 = \mu_3 = 10^{-3}$, $\eta_L = \eta_\Psi = 0.5$ (relative)	A

[Deviation] Arrival rate. With 1.2 MB tasks, Table IV's 6–20 tasks/s per IoT means 58–190 Mbit/s of demand per device, against an uplink of a few Mbit/s and a local CPU that finishes about 0.45 tasks/s. Every policy is then hopelessly overloaded. The paper's Fig. 8b, where domains 3, 4 and 6 complete almost all generated tasks, implies a load close to capacity, so we scale λ by 0.05. Our LEO rates (0.5–5 Mbit/s) match the "hundreds of kbit/s to several Mbit/s" stated in the paper.

6.2 Task stream and protocol

Each seed samples four tasks per domain (tasks 1–24, numbered domain by domain). A task differs from the others in its IoT positions, arrival rates, task sizes, processing densities, starting UAV positions and offset within the LEO window. One *iteration* visits all 24 tasks in random order; each visit runs one 120-slot episode with all IoTs of the task sharing the task's policy, followed by one learning update. Learning curves record the penalty of that on-policy, exploring episode. All four learners use identical tasks. Main results average three seeds (0, 1, 2).

6.3 Evaluation metrics

- **Penalty** (15), **AoI** and **scaled energy** ϵE, averaged per IoT and per slot.
- **Jumpstart**: the reduction in initial penalty on an unseen task relative to MSR-NAC from a random initialisation, after 15 pre-training rounds.
- **Convergence speedup**: the iteration at which a 10-iteration moving average first reaches NAC's final level (mean of its last 20 smoothed iterations), divided into NAC's own iteration; NAC therefore scores 1.
- **Forgetting**: the penalty on old tasks 1–24 before and after learning new tasks 25–28.

Chapter 7

Results and Discussion

Every number in this chapter comes from `experiments/results/*.json` produced by the code in Chapter 5. Paper values are quoted from Wu et al. [1] and labelled as such. Unless stated otherwise, results are means over three seeds.

7.1 Convergence in domain 6 (Fig. 3 of the paper)

Figure 7.1 shows the learning curves for the four domain-6 tasks, averaged over three seeds. All four learners improve, but slowly. After 200 iterations independent MSR-NAC is lowest (17.1), followed by MSR-NAC-ELLA (18.2), MSR-NAC-CDLL (19.6) and NAC (19.8). The dashed line is the best stationary decision found by the exhaustive grid, evaluated without exploration noise (Section 7.4).

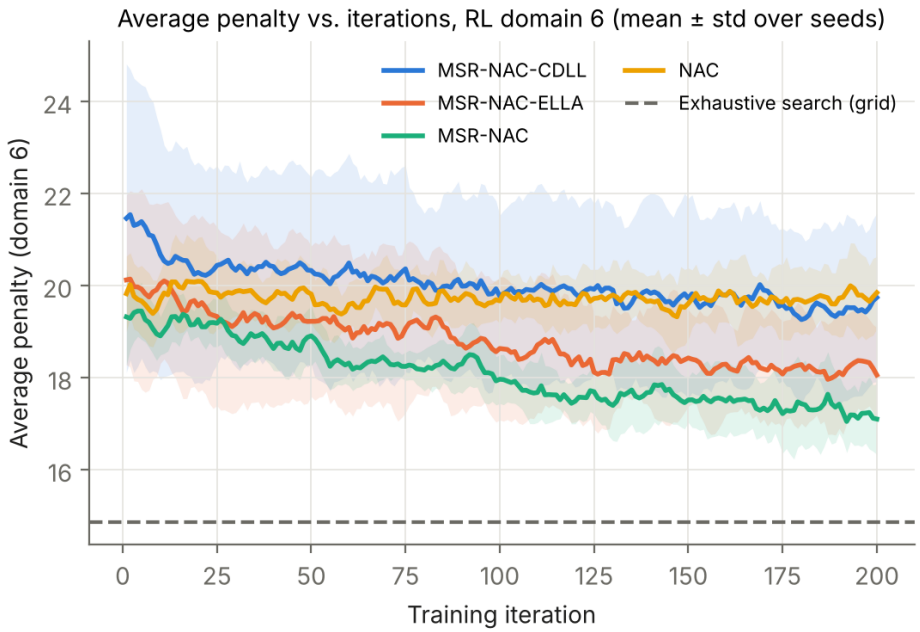


Figure 7.1: Ours: average penalty vs. iteration on the domain-6 tasks (mean ± std over 3 seeds, 5-iteration moving average).

The paper's Fig. 3 shows CDLL starting near 100 and NAC near 320, with CDLL converging

fastest. We do not see this. In our runs CDLL starts *higher* than the others in domain 6 (21.4 vs. about 19.5–20.1 over the first five iterations). When the domain-6 tasks are first visited, the projection Ψ^6 is still random and the warm-start code is borrowed from the other domains, so the reconstructed policy is effectively random. The baselines start from $\theta \sim U(0, 1)$, which in our parameterisation happens to be a reasonable policy.

7.2 Per-domain penalty, AoI and energy (Fig. 4 of the paper)

Figure 7.2 and Tables 7.1 to 7.3 average every training iteration, as Fig. 4 of the paper does, so they include the early, poorly trained part of each curve. Table 7.4 shows the end of training.

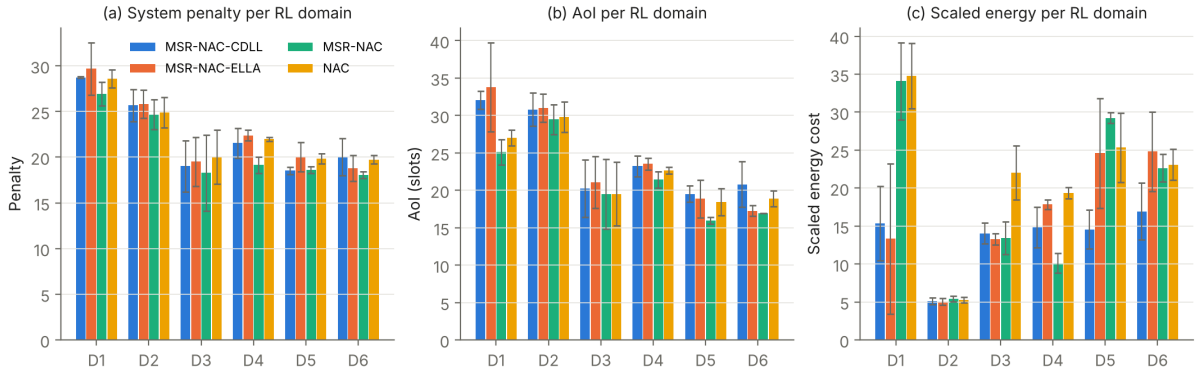


Figure 7.2: Ours: average penalty, AoI and scaled energy per domain over 200 iterations (error bars: std over seeds).

Table 7.1: Average penalty over the 200 training iterations (mean over 3 seeds; lowest per domain in bold).

Domain	MSR-NAC-CDLL	MSR-NAC-ELLA	MSR-NAC	NAC
D1	28.73	29.68	**26.92**	28.57
D2	25.66	25.80	**24.66**	24.90
D3	19.01	19.50	**18.29**	20.03
D4	21.54	22.39	**19.15**	21.98
D5	**18.53**	20.01	18.60	19.81
D6	20.02	18.78	**18.07**	19.74

Table 7.2: Average AoI (slots) over the 200 training iterations.

Domain	MSR-NAC-CDLL	MSR-NAC-ELLA	MSR-NAC	NAC
D1	32.08	33.77	25.12	27.02
D2	30.80	31.00	29.48	29.81
D3	20.25	21.06	19.50	19.52
D4	23.23	23.52	21.41	22.63
D5	19.52	18.85	15.94	18.43
D6	20.79	17.27	16.93	18.90

Table 7.3: Average scaled energy over the 200 training iterations.

Domain	MSR-NAC-CDLL	MSR-NAC-ELLA	MSR-NAC	NAC
D1	15.33	13.32	34.10	34.77
D2	5.10	5.03	5.40	5.25
D3	14.02	13.27	13.42	22.04
D4	14.80	17.84	10.11	19.37
D5	14.57	24.62	29.24	25.33
D6	16.94	24.83	22.64	23.10

Table 7.4: Penalty at the end of training (mean of the last 10 iterations, 3 seeds).

Algorithm	D1	D2	D3	D4	D5	D6	All
MSR-NAC-CDLL	27.81	25.45	18.74	20.12	**18.16**	19.60	21.65
MSR-NAC-ELLA	29.17	25.55	18.73	22.13	19.58	18.22	22.23
MSR-NAC	**26.40**	**24.53**	**18.22**	**18.20**	18.41	**17.12**	**20.48**
NAC	28.26	25.04	19.89	21.69	20.36	19.83	22.51

What reproduces.

- **MSR-NAC beats NAC in every domain**, by 6.9% on average over training (20.95 vs. 22.50) and by 9.0% at the end (20.48 vs. 22.51). This matches the paper's claim that n-step returns make the base learner more accurate and more stable.
- **CDLL beats the single-domain lifelong baseline** in five of six domains (all except domain 6), and beats NAC in domains 3, 4 and 5.
- **The domain ordering agrees with the paper:** domain 1 (local computing only) has the worst penalty for every learner, domains 1 and 2 have the highest AoI, domain 2 has the lowest energy, and the offloading domains 3–6 have the lowest penalties.
- **CDLL saves energy.** It uses the least energy of all four learners in domains 5 and 6 (14.6 and 16.9, against 22.6–29.2 for the others), and far less than MSR-NAC and NAC in domain 1.

What does not reproduce. Independent MSR-NAC, which shares no knowledge, has the lowest penalty in five of six domains, both averaged over training and at the end. The exception is domain 5, where CDLL is lowest. Overall CDLL is second at the end of training (21.65 vs. 20.48 averaged over domains). Its energy savings come with higher AoI (e.g. 20.8 vs. 16.9 in domain 6), and at $\beta = 0.8$ the AoI term dominates. The paper's "up to 70%" reduction is not observed: our largest gap between any two methods in any domain is about 18% (domain 4 at the end of training, MSR-NAC vs. ELLA).

7.3 Convergence speed (Fig. 7 of the paper)

Table 7.5: Convergence speedup relative to NAC (NAC = 1 by definition).

Algorithm	D1	D2	D3	D4	D5	D6	Mean
MSR-NAC-CDLL	0.36	0.35	1.66	0.22	0.69	0.41	0.61
MSR-NAC-ELLA	1.03	0.02	4.72	3.12	0.34	0.78	1.67
MSR-NAC	2.78	3.50	2.99	2.25	1.00	2.26	2.46
NAC	1.00	1.00	1.00	1.00	1.00	1.00	1.00

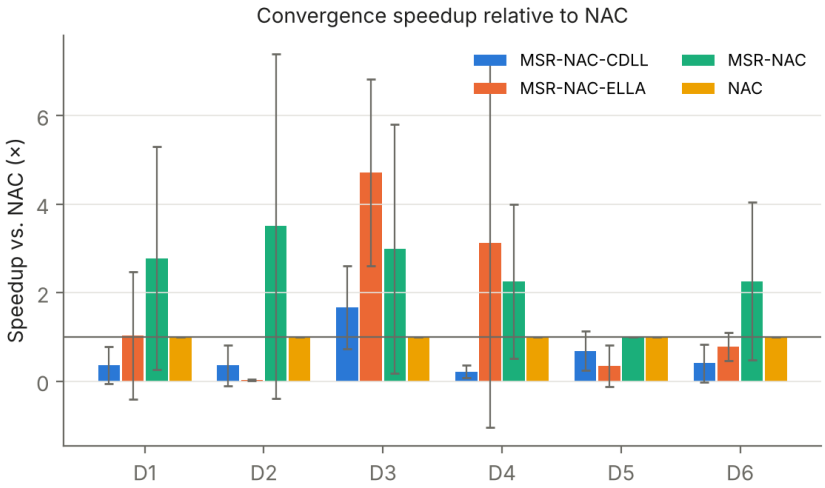


Figure 7.3: Ours: convergence speedup relative to NAC (iterations needed to reach NAC's final penalty level).

MSR-NAC reaches NAC's final penalty 2.5× sooner on average, and ELLA 1.7× sooner. CDLL averages 0.61×: in most domains it reaches NAC's final level only near the end of training, and in domain 3 it gets there faster (1.7×). The paper reports an average speedup above 8× for CDLL. We do not reproduce it.

7.4 Exhaustive-search reference

For the four domain-6 tasks of seed 0, the grid of 960 stationary decisions gives a mean penalty of 14.9 (per task: 14.8, 16.0, 16.8, 11.9; AoI 12.96–17.75), re-evaluated on three fresh episodes without exploration noise. The best decisions mix executors (for example 25% local and 75% LEO, half CPU, half remote CPU, full power). The learned policies end at 17.1–19.8 on the exploring roll-outs. Part of this gap is the exploration noise, since the grid is evaluated deterministically; part is a genuine optimisation gap after 200 iterations. The paper also reports that its learners come close to exhaustive search.

7.5 AoI–energy trade-off versus β (Fig. 5 of the paper)

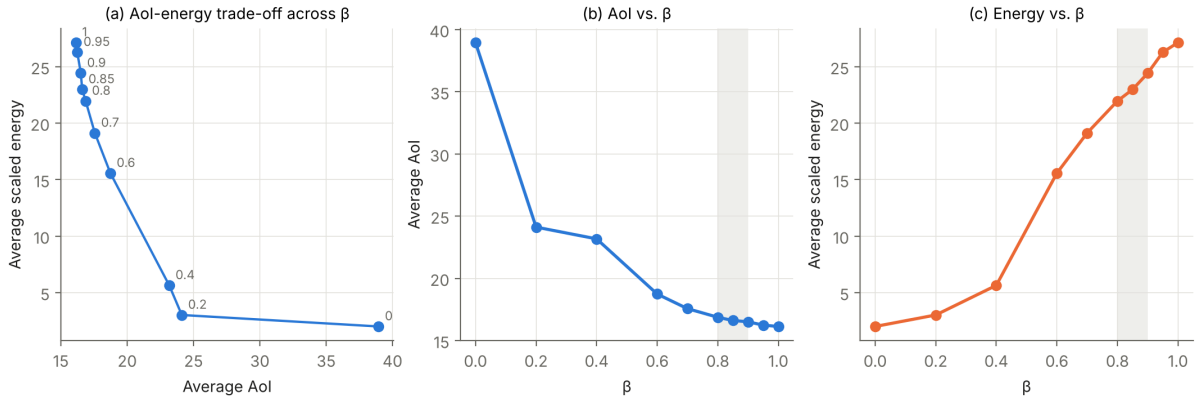


Figure 7.4: Ours: MSR-NAC-CDLL trained on the domain-6 tasks for each β (100 iterations, seed 0). Shaded band: the paper's recommended range $\beta \in [0.8, 0.9]$.

Table 7.6: AoI and scaled energy of MSR-NAC-CDLL on the domain-6 tasks for different β.

β	AoI	Energy	Penalty
0.0	38.95	2.00	2.00
0.2	24.12	3.01	7.23
0.4	23.17	5.63	12.65
0.6	18.73	15.57	17.47
0.7	17.56	19.13	18.03
0.8	16.87	21.96	17.89
0.85	16.61	22.98	17.56
0.9	16.50	24.42	17.30
0.95	16.23	26.28	16.74
1.0	16.13	27.13	16.13

As β goes from 0 to 1, AoI falls monotonically from 38.95 to 16.13 and energy rises from 2.0 to 27.1, which is the expected trade-off curve. Our AoI at $\beta = 0$ (38.95) is very close to the paper's 38.58. Two of the paper's observations do not reproduce. First, the paper finds a "win–win" region in which raising β from 0 to 0.85 lowers *both* AoI and energy; in our runs energy rises over that range. Second, the paper sees energy jump 5.5× between $\beta = 0.85$ and $\beta = 1$, while ours rises only 18%. Our curve is steepest at low β: most of the AoI reduction (38.9 to 18.7) has happened by $\beta = 0.6$, and beyond $\beta \approx 0.8$ more weight on AoI buys little. This makes $\beta = 0.8$ a sensible default in our setting as well.

7.6 Jumpstart on unseen tasks (Fig. 6 of the paper)

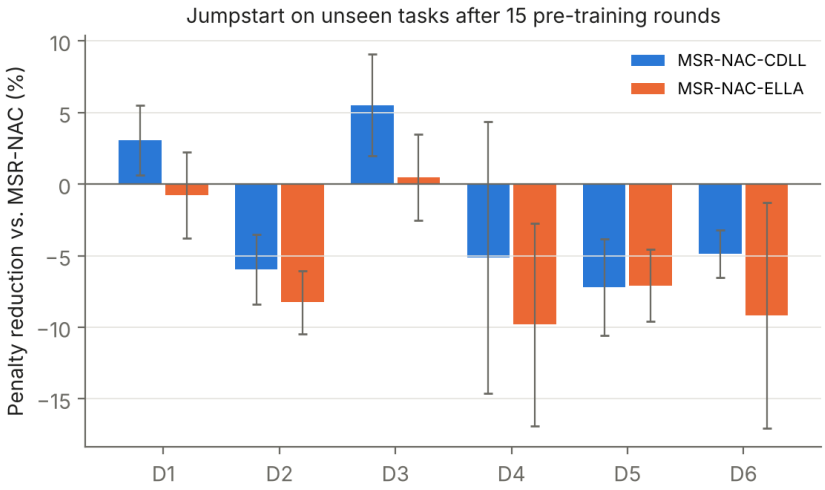


Figure 7.5: Ours: initial-penalty reduction relative to MSR-NAC (random initialisation) after 15 pre-training rounds on tasks 1–24.

Table 7.7: Jumpstart: initial-penalty reduction (%) on unseen tasks relative to MSR-NAC from a random start.

Algorithm	D1	D2	D3	D4	D5	D6	Mean
MSR-NAC-CDLL	3.1	-6.0	5.5	-5.1	-7.2	-4.9	-2.4
MSR-NAC-ELLA	-0.8	-8.3	0.5	-9.8	-7.1	-9.2	-5.8

The paper reports that CDLL starts 27–67% lower than MSR-NAC on a new task. We find no positive jumpstart on average: CDLL's warm-start policy is better than a random start in domains 1 and 3 (+3.1% and +5.5%) but worse in the other four, for a mean of −2.4%. CDLL does beat the single-domain ELLA baseline (mean −5.8%) in every domain except domain 5, where they tie, which is consistent with the paper's claim that cross-domain sharing transfers more than within-domain sharing. The main reason for the missing jumpstart is the one discussed in Chapter 8: in our parameterisation a random $\theta \sim U(0, 1)$ already gives a reasonable policy, so a warm start has little room to help.

7.7 System analysis with the trained CDLL policy (Fig. 8 of the paper)

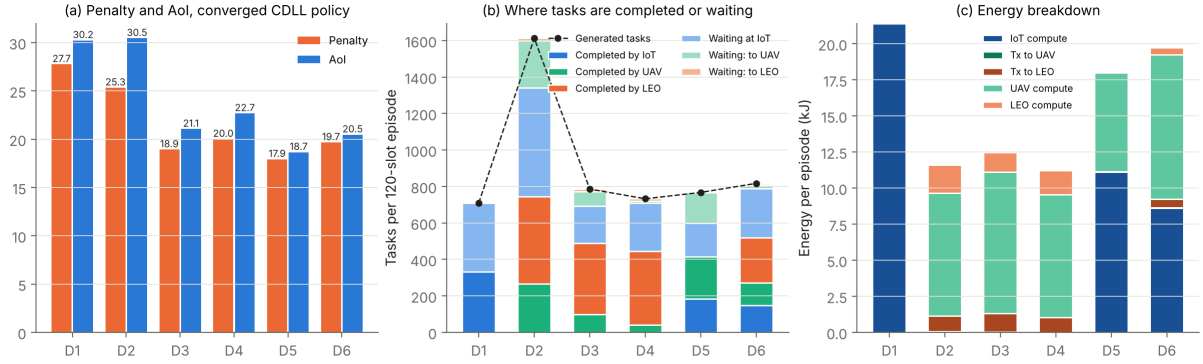


Figure 7.6: Ours: penalty and AoI, task distribution and energy breakdown per domain with the trained CDLL policies (5 evaluation episodes per task, 3 seeds).

Table 7.8: System analysis with the trained CDLL policies (per 120-slot episode; tasks generated and completed by executor).

Domain	Penalty	AoI	Energy	Generated	IoT	UAV	LEO
D1	27.7	30.2	17.8	709	331	0	0
D2	25.3	30.4	4.8	1613	0	265	481
D3	18.9	21.1	10.4	786	0	100	388
D4	20.0	22.7	9.4	733	0	41	404
D5	17.9	18.7	15.0	768	183	232	0
D6	19.7	20.5	16.5	817	148	123	249

The qualitative picture matches the paper's Fig. 8. In domain 1, only about half the generated tasks finish within the episode and the rest wait at the IoTs, giving the highest AoI. Domain 2 generates twice as many tasks (20 IoTs) and has the lowest energy per IoT because it cannot compute locally. In domains 3 and 4 the LEO satellites complete most of the work (about 80–90% of completed tasks), which matches the paper's Fig. 8b, where LEOs complete roughly two thirds of the tasks in domain 3. In domain 5 the UAVs take the largest share, and in domain 6 the satellites take over half of the remote work.

The paper also reports that adding satellites to domain 5 (making domain 6) lowers penalty and AoI (61.9/49.4 to 48.5/45.1). In our runs domain 6 is slightly *worse* (19.7/20.5 vs. 17.9/18.7). The domain-5 and domain-6 tasks are different random tasks, and our domain-6 tasks carry about 6% more load, so this is not a controlled comparison in either the paper or our study.

7.8 Scalability and control-plane overhead (Table V of the paper)

Table 7.9: Scalability with $U = N/5$ UAVs and 18 LEOs over 600 slots (3 seeds). Control-plane figures are modelled, not measured.

N	AoI	Energy/slot	Energy/IoT/slot	Ctrl. energy	Ctrl. latency (ms)	Sim. time (s)
80	98.07	875.3	10.941	0.0040%	0.213	0.8
120	89.09	1303.9	10.866	0.0042%	0.233	1.2
160	93.51	1682.4	10.515	0.0041%	0.254	2.1
200	86.38	2043.2	10.216	0.0043%	0.274	2.5
240	87.15	2429.0	10.121	0.0043%	0.295	3.0
280	87.91	2763.2	9.868	0.0043%	0.315	3.9

As in the paper, growing the network from 80 to 280 IoTs (with $U = N/5$ UAVs and the same 18 satellites) leaves the policy size unchanged at 15 parameters, AoI roughly flat (98.1 to 87.9), and energy per IoT slightly lower (10.94 to 9.87, -10%; the paper reports -29%). Total energy grows 3.2× for a 3.5× increase in devices (paper: 2.48×). Absolute AoI differs strongly from the paper (about 90 vs. about 3): the per-IoT arrival rate we calibrated for 10 IoTs overloads the shared satellites when 280 IoTs use them, while the paper's Table V scenario is evidently lightly loaded. The control-plane numbers come from a message-size model: 32-byte status reports per IoT per slot and policy dissemination of $4d^g$ bytes per learning round. They show signalling energy far below 1% of data-transmission energy and sub-millisecond latency, consistent with the paper's conclusion that control overhead is small, but they are not measurements.

7.9 Continual learning and forgetting (Fig. 9 of the paper)

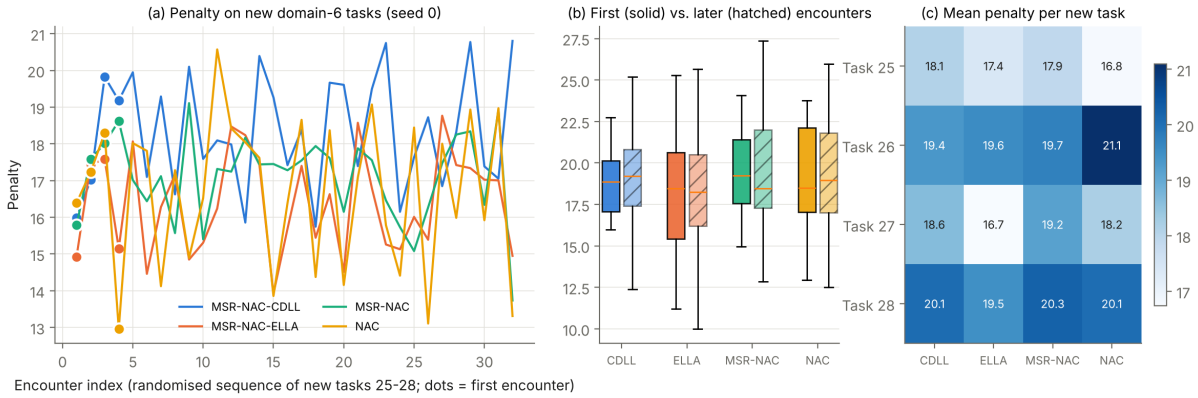


Figure 7.7: Ours: learning four new domain-6 tasks (25–28) in a randomised sequence of 32 encounters after pre-training on tasks 1–24.

Table 7.10: Continual learning on new domain-6 tasks 25–28 and forgetting on old tasks 1–24 (mean penalty, 3 seeds).

Algorithm	New: first enc.	New: later enc.	Old: before	Old: after
MSR-NAC-CDLL	18.57	19.10	21.62	21.61
MSR-NAC-ELLA	18.56	18.26	22.20	22.19
MSR-NAC	19.43	19.23	20.48	20.48
NAC	19.01	19.03	22.52	22.52

On a task's *first* encounter, both lifelong learners start lower (CDLL 18.57, ELLA 18.56) than MSR-NAC (19.43) and NAC (19.01): knowledge from tasks 1–24 does help new tasks of a known domain. On later encounters ELLA is best (18.26), with CDLL (19.10) close to the independent learners. Forgetting is negligible for every method: the mean penalty on tasks 1–24 changes by at most 0.01 after the new tasks are learned. For the lifelong learners this is the paper's claim that the KB does not forget. For the independent learners it holds by construction, because they store one parameter vector per task and never touch the old ones, at the cost of memory that grows linearly with the number of tasks.

7.10 Ablation study

Table 7.11: Ablations (mean penalty over training and over the last 10 iterations, seed 0, 100 iterations).

Variant	Mean	Final
n_step=1	22.53	23.75
n_step=10	18.75	18.62
n_step=40 (default)	18.99	19.21
n_step=120 (MC)	18.93	18.67
CDLL h=4	23.70	23.40
CDLL h=8 (default)	22.52	22.19
CDLL h=15	22.75	22.96
CDLL s_k=rand (Alg.1)	22.62	21.77
CDLL no refit	24.35	25.63

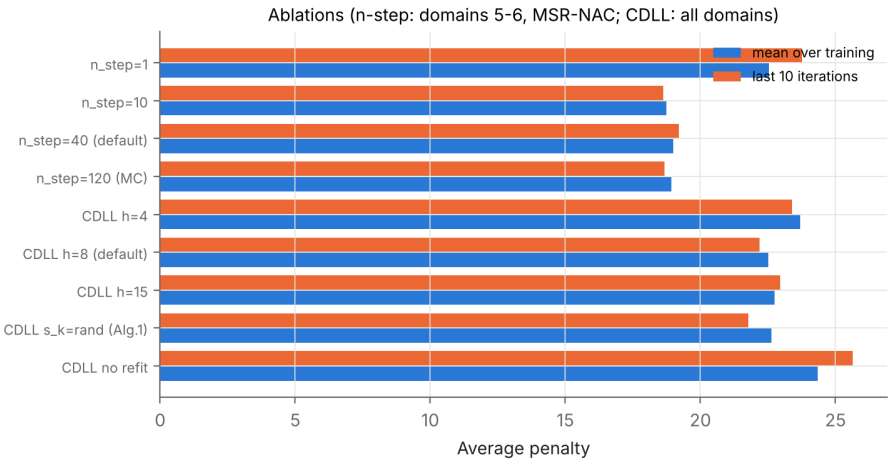


Figure 7.8: Ours: ablations.

The ablations (Table 7.11) used seed 0 and 100 iterations. The n-step variants train MSR-NAC on domains 5–6; the CDLL variants train on all six domains.

- **Multi-step returns matter.** With $n = 1$ (one-step bootstrapping) the penalty is 22.5 averaged over training, against 18.7–19.0 for $n \in \{10, 40, 120\}$. Credit for an offloading decision arrives many slots later, when the task completes, and a one-step target cannot carry it back. This supports the paper's motivation for MSR-NAC. The differences between $n = 10$, 40 and Monte-Carlo returns are within noise.

- **Knowledge-base size.** $h = 8$ (our default) beats both $h = 4$ (23.7), which is too small to represent the 24 policies, and $h = 15$ (22.7), which shares less.

- **Refitting $\mathbf{s}^k$ is essential.** Updating only the current task's code, as Algorithm 1 is written, raises the penalty from 22.5 to 24.4 (final 22.2 to 25.6). Each change to the shared $\mathbf{L}$ moves the reconstructed policies of every other task.

- **Warm start does not help.** Initialising a new task's code randomly, as in Algorithm 1, gives the same training average (22.6 vs. 22.5) and a *better* final penalty (21.8 vs. 22.2). Our warm-start deviation therefore does not explain any CDLL advantage, and Algorithm 1's original choice is at least as good.

Chapter 8

Comparison with the Paper and Limitations

8.1 Side-by-side comparison

Table 8.1: The paper's claims against our measurements.

Claim	Paper	Ours	Verdict
Mean visible LEOs after screening	2.73	2.73	reproduced
MSR-NAC beats NAC in every domain	yes	yes; -6.9% over training, -9.0% at end	reproduced
CDLL beats single-domain ELLA	most domains	5 of 6 domains (training average)	reproduced
CDLL best in the complex domains 5–6	about half NAC's penalty	best in D5 only (-6% vs. NAC); MSR-NAC best in D6	partly
Up to 70% lower AoI/energy	up to 70%	largest gap between methods about 18%	not reproduced
Convergence speedup vs. NAC	$> 8\times$ (CDLL)	$0.61\times$ (CDLL), $2.5\times$ (MSR-NAC)	not reproduced
Jumpstart vs. MSR-NAC	27–67%	-2.4% (CDLL), -5.8% (ELLA)	not reproduced
AoI vs. β	38.58 at β=0 to 20.54 at β=1	38.95 to 16.13	reproduced
Win–win region $\beta \in [0, 0.85]$	AoI and energy both fall	energy rises	not reproduced
LEOs help (domain $5 \to 6$)	penalty $61.9 \to 48.5$	$17.9 \to 19.7$	not reproduced
Scaling $80 \to 280$ IoTs	AoI $+3\%$, energy/IoT -29%	AoI -10%, energy/IoT -10%	trend reproduced
No catastrophic forgetting	qualitative	old-task penalty change ≤ 0.01	reproduced

8.2 Why our numbers differ

We see five plausible reasons. None of them can be confirmed without the authors' code.

1. **Unstated learning details.** The paper gives no learning rates, no h or d_{base}, no n for MSR-NAC, no number of trajectories per update and no policy parameterisation. These

settings decide how quickly any learner improves. We chose them once, for stable learning, and used them for every method. A configuration tuned for CDLL could change the ranking. Our ablations (Section 7.10) show that h and the refit of $\mathbf{s}^k$ matter.

2. **Different load regime.** Taken literally, Table IV overloads every node about a hundred times over. We rescaled the arrival rate so that the load is near capacity. In a near-capacity regime the advantage of a good policy over a mediocre one is bounded; in the paper's regime, absolute penalties are larger and differences between methods may be larger too.

3. **Initial policies.** The paper's Fig. 3 has NAC starting near 320 while CDLL starts near 100, so its baselines must begin from very poor policies. Ours begin from $\theta \sim U(0, 1)$, which already produces sensible behaviour (penalty about 20). Much of the paper's "70%" and "8×" gap is measured during this initial phase. A strong baseline initialisation leaves little room for a jumpstart.

4. **The KB executes reconstructed policies.** CDLL and ELLA never run the policy the base learner just produced. They run its projection onto the knowledge base, $\Psi^g \mathbf{L}\mathbf{s}^k$. Our reconstruction error stays around 20%, so part of every natural-gradient step is lost, and learning slows compared with running MSR-NAC alone. This is a known cost of PG-ELLA-style methods when tasks are not low-rank.

5. **Stochastic, on-policy curves.** All curves include exploration noise and a different random episode at every iteration, so differences of one or two penalty units lie within seed-to-seed variation (see the error bars).

8.3 Limitations of this study

- Simulation only. The channel, energy-scale and policy parameters marked as assumed affect the absolute numbers.
- Three seeds per configuration. Confidence intervals are wide relative to some of the differences.
- The exhaustive baseline searches stationary decisions on a grid. It is a reference point, not the true optimum over state-dependent policies.
- The control-plane figures of Table V are computed from a message-size model, not measured on an SDN testbed.
- Hyperparameters were not tuned separately for each method (deliberately, to avoid favouring one method), so every method may be below its best.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

This project re-implemented, from the equations alone, the KB-SAT-SDN system model and the CDLL-SATECN algorithm of Wu et al. [1]. The implementation includes a slotted simulator driven by real Starlink ephemerides, the six RL domains with policy dimensions from 3 to 15, the NAC and MSR-NAC base learners, a single-domain lifelong baseline, the cross-domain knowledge base of Eqs. (20)–(25), and an exhaustive-search reference. All of the paper's experiments were re-run with three seeds, together with ablations, and five unit tests check the model and the mathematics.

The reproduction confirms several of the paper's findings: the satellite-visibility statistic (2.73), the advantage of multi-step returns over plain NAC in every domain, the advantage of cross-domain over single-domain lifelong learning in most domains, the expected AoI–energy trade-off as β varies, the qualitative per-domain behaviour, the near-flat AoI as the network grows, and the absence of catastrophic forgetting. It does not confirm the size of CDLL's advantage. In our runs independent MSR-NAC learns as well as or better than CDLL in five of six domains, there is no positive jumpstart on unseen tasks, and CDLL is not faster to converge than NAC. We attribute the gap mainly to learning details the paper does not state (parameterisation, step sizes, basis size, initial policies) and to the cost of always executing the knowledge base's reconstruction instead of the freshly improved policy. Everything needed to test these explanations, including the configuration and the ablation hooks, is in the code.

9.2 Future scope

- **Tune for transfer.** Search h, d_{base}, μ_1 and the KB step sizes on held-out tasks, and let each task take several MSR-NAC steps before the KB update, as in the original PG-ELLA, to reduce the reconstruction loss.
- **Weaker initial policies**, to measure jumpstart in the regime the paper's Fig. 3 suggests.
- **Controlled domain comparisons**: evaluate the same task parameters under domains 5 and 6 to isolate the value of the LEO layer.

- The paper's own future directions: UAV trajectory and battery dynamics in the decision process, and secrecy constraints on power and bandwidth.
- A packet-level SDN emulation (e.g. Mininet with an ONOS/Ryu controller) to *measure* the control-plane overhead that we only modelled.

Bibliography

[1] Yinxuan Wu, Ning Wang, Beatriz Lorenzo, Sumudu Samarakoon, and Bing Liu. Lifelong learning-based SDN design for dynamic configuration and resource allocation in satellite-terrestrial networks. *IEEE Transactions on Wireless Communications*, 2026. doi: 10.1109/TWC.2026.3716226. Accepted, early access.

[2] Sanjit Kaul, Roy Yates, and Marco Gruteser. Real-time status: How often should one update? *Proc. IEEE INFOCOM*, pages 2731–2735, 2012.

[3] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526, 2017.

[4] Haitham Bou Ammar, Eric Eaton, Paul Ruvolo, and Matthew Taylor. Online multi-task learning for policy gradient methods. In *Proc. 31st International Conference on Machine Learning (ICML)*, volume 32 of *PMLR*, pages 1206–1214, 2014.

[5] X. Xia, H. H. Esmat, K. Dyer, B. Lorenzo, and L. Guo. Cross-domain federated computation offloading for age of information minimization in satellite-airborne-terrestrial networks. *Proc. IEEE PIMRC*, pages 1–7, 2023.

[6] M. Liao, R. Wang, P. Zhang, and Z. Xian. Information freshness optimal resource allocation for LEO-satellite internet of things. *IEEE Internet of Things Journal*, 11(10):17372–17387, 2024.

[7] Zhuoxing Zhu, Shaohua Wan, Pingzhi Fan, and Khaled B. Letaief. Federated multiagent actor-critic learning for age sensitive mobile-edge computing. *IEEE Internet of Things Journal*, 9(2):1053–1067, 2022.

[8] Diego Kreutz, Fernando M. V. Ramos, Paulo E. Veríssimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76, 2015.

[9] Yongpeng Shi, Yurui Cao, Jiajia Liu, and Nei Kato. A cross-domain SDN architecture for multi-layered space-terrestrial integrated networks. *IEEE Network*, 33(1):29–35, 2019.

[10] Chen Chen, Zhuoyu Liao, Ying Ju, Chang He, Keping Yu, and Shaohua Wan. Hierarchical domain-based multicontroller deployment strategy in SDN-enabled space–air–ground integrated network. *IEEE Transactions on Aerospace and Electronic Systems*, 58(6):4864–4879, 2022.

[11] Haitham Bou Ammar, Eric Eaton, José Marcio Luna, and Paul Ruvolo. Autonomous cross-domain knowledge transfer in lifelong policy gradient reinforcement learning. In *Proc. 24th International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.

[12] Yinxuan Wu, Beatriz Lorenzo, and Bing Liu. Lifelong learning for AoI and energy trade-off optimization in satellite-airborne-terrestrial edge computing networks. In *Proc. IEEE PIMRC*, pages 1–7, 2023.

[13] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[14] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *Proc. ICLR*, 2019.

[15] Akram Al-Hourani, Sithamparanathan Kandeepan, and Simon Lardner. Optimal LAP altitude for maximum coverage. *IEEE Wireless Communications Letters*, 3(6):569–572, 2014.

[16] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.

[17] Jan Peters and Stefan Schaal. Natural actor-critic. *Neurocomputing*, 71(7):1180–1190, 2008.

[18] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proc. AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[19] Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

[20] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[21] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[22] Brandon Rhodes. sgp4: Track earth satellites given TLE data. `https://pypi.org/project/sgp4/`, 2024.

[23] David A. Vallado, Paul Crawford, Richard Hujsak, and T. S. Kelso. Revisiting spacetrack report #3. *Proc. AIAA/AAS Astrodynamics Specialist Conference*, 2006.

[24] CelesTrak. Starlink GP element sets (TLE format). `https://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&FORMAT=tle`, 2026. Snapshot retrieved 8 November 2026.

Appendix A

Reproducing the Results

```
git clone https://github.com/AnsFahad/satecn-cdll.git
cd satecn-cdll
python -m venv .venv
.\.venv\Scripts\Activate.ps1 # Windows PowerShell
pip install -r requirements.txt
python -m pytest -q # 5 unit tests
python scripts/run_experiments.py --exp all --workers 4
python scripts/make_figures.py # figures -> experiments/graphs
```

A smoke test (`-quick`) runs every experiment at reduced size in a few minutes.